

Introduction to Interrupts

Ankit Srivastava

EE2028

ARMv7-M Architecture Ref Manual

Figures from “The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors” by Joseph Yiu

Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

- Introduction to Interrupts
- Polling vs Interrupt
- Interrupt Processing

INTRODUCTION TO INTERRUPTS

- A computer system normally connects to I/O and other devices
 - keyboard
 - monitor
 - mouse
 - sensors etc.
- Processor needs to serve these devices at unpredictable times (e.g., CPU does not know when you are about to hit a key)



POLLING VS INTERRUPT

- Imagine processor is like a CEO working in his office, and devices are guests
 - Polling is like the CEO who goes to open the door periodically to see if a guest is there
 - Interrupt is like when a guest comes, he/she knocks on the door. CEO then stops what he/she is doing and prepares to welcome the guest



POLLING VS INTERRUPT

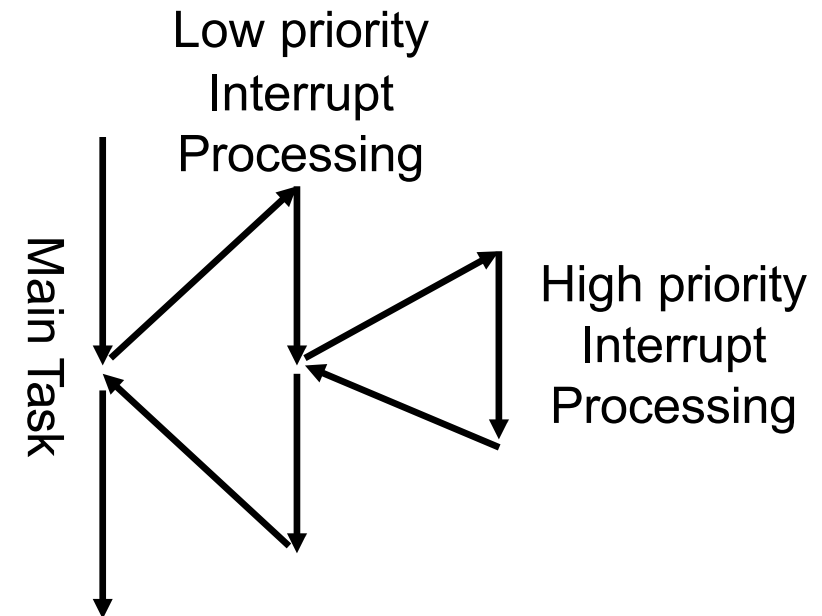
- Alternative 1: Polling
 - Periodically processor checks each device to see if it has a request
 - Takes CPU time even if no request is pending
 - How does software know when to let the system poll?
- Alternative 2: Interrupts
 - Give each device a wire (interrupt line) that it can use to signal the processor
 - When interrupt signalled, processor executes a routine called an interrupt service routine (ISR) to deal with the interrupt
 - No overhead when no requests pending

POLLING VS INTERRUPT

- Polling
 - (+) CPU determines how often it needs to poll. Potentially simpler software
 - (-) CPU needs to check the device many times, and in most cases, the device might not have new data / situation which require attention from CPU
 - (-) Most I/O devices operate at a much slower speed than CPU
- Interrupt
 - (+) More efficient since CPU can do other work
 - (-) Harder to design, debug and test
 - (-) Overhead/latency : CPU has to stop working, determine which device is requesting service, save its work before serving the request

INTERRUPT PROCESSING

- CPU is executing the main task
- A device/peripheral requests for service using an interrupt
- Based on the interrupt configuration and priority, CPU decides if it allows that interrupt
- If yes, save all the status (R0-R3, SP, LR, PC, APSR saved automatically, others may need manual saving) onto stack, run the interrupt handler (a function that is invoked by the processor hardware automatically) to serve the I/O device
- When done, CPU reloads all status as it was before processing interrupt, and continues the main task
- A higher priority interrupt can interrupt or pre-empt the lower priority one (interrupt nesting)





CONCLUSION

- Introduction to Interrupts
- Polling vs Interrupt
- Interrupt Processing